

Revisão de Exercícios MongoDB

<https://gemini.google.com/app/1b63f7bf4d6e2029>

User prompt: Pode verificar se as minhas respostas estão corretas?? Ex 1 a 6, vou mandar o restante depois, verifique esses

Response: Todas as consultas atendem à lógica proposta, com apenas uma observação importante de escopo no Exercício 6 e um detalhe cosmético no Exercício 3:

Exercício 1

- **Status:** Correto.
- **Comentário:** A sintaxe `db.pedidos.find({ QUANTITYORDERED: { $gt: 50 } })` aplica perfeitamente o operador `$gt` no campo numérico para filtrar pedidos com quantidade superior a 50.

Exercício 2

- **Status:** Correto.
- **Comentário:** A projeção `db.pedidos.find({}, { ORDERNUMBER: 1, SALES: 1, STATUS: 1, _id: 0 })` inclui apenas os campos solicitados e remove o identificador padrão com `_id: 0`.

Exercício 3

- **Status:** Correto na lógica.
- **Comentário:** O encadeamento do `$match` com intervalo de datas seguido pelo agrupamento global `_id: null` somando `"$SALES"` está impecável.
- **Ajuste sugerido:** O campo de saída foi nomeado como `totalUnidades`, provavelmente herdado do exemplo da apostila. Como o cálculo envolve valores monetários (`$SALES`), um alias como `totalSales` ou `totalVendas` representaria melhor o dado.

Exercício 4

- **Status:** Correto.
- **Comentário:** A agregação `db.pedidos.aggregate([ { $group: { _id: "$COUNTRY", totalCountry: { $sum: "$SALES" } } } ])` agrupa perfeitamente por país e calcula a soma das vendas por chave.

Exercício 5

- **Status:** Correto e com boa prática.
- **Comentário:** A média foi calculada corretamente usando `_id: null` e `$avg: "$SALES"`. O estágio extra com `$project` e `$round` para formatar o resultado em duas casas decimais demonstrou ótimo domínio da ferramenta.

Exercício 6

- **Status:** Atenção ao enunciado.
- **Comentário:** A ordenação `.sort({ SALES: -1 })` está certa. Porém, o enunciado pediu para recuperar **todos os pedidos** (isto é, os documentos completos).
- **Ponto de correção:** Ao passar a projeção `{ _id: 0, ORDERNUMBER: 1, PRODUCTLINE: 1, SALES: 1 }`, você omitiu campos essenciais do pedido (como `datas`, `cliente` e `status`). Para atender estritamente ao exercício, basta remover a projeção:

```
db.pedidos.find().sort({ SALES: -1 })
```

Pode enviar os exercícios de 7 a 12 quando estiver pronto para a próxima rodada de conferência.

User prompt: ex 07 a 12

Response: Todas as operações e lógicas aplicadas nos exercícios 7 a 12 estão corretas, demonstrando excelente domínio dos operadores de atualização, remoção e filtros por texto/data:

Exercício 7

- **Status:** Correto.
- **Comentário:** A sintaxe com `updateOne`, o filtro `{ ORDERNUMBER: 10107 }` e o operador `{ $set: { STATUS: "Delivered" } }` foi aplicada com precisão. A consulta de verificação em seguida comprovou a alteração do valor.
- **Observação técnica:** O `modifiedCount: 0` no retorno apenas indica que o registro já continha o valor "Delivered" no momento da execução (caso o comando tenha rodado mais de uma vez).

Exercício 8

- **Status:** Correto.
- **Comentário:** O comando `db.pedidos.updateMany({ QUANTITYORDERED: { $lt: 30 } }, { $set: { STATUS: "Processing" } })` utilizou corretamente o operador relacional `$lt` junto com `$set`, atualizando todos os 9 documentos esperados.

Exercício 9

- **Status:** Excelente.
- **Comentário:** Seguiu à risca as boas práticas e o enunciado: fez a checagem prévia com `countDocuments`, executou o `deleteMany({ COUNTRY: "USA" })` e realizou a validação posterior confirmando a contagem zerada.

Exercício 10

- **Status:** Correto na lógica de busca.
- **Comentário:** O uso de `$regex: "Young"` combinado com `$options: "i"` cobre variações de maiúsculas/minúsculas de forma adequada.
- **Ponto de atenção:** Assim como no Exercício 6, você aplicou uma projeção no segundo parâmetro do `find()` (`{ _id: 0, ORDERNUMBER: 1, ... }`). Para fins didáticos e legibilidade no terminal funciona muito bem, mas se o corretor/avaliador exigir os documentos completos conforme o enunciado ("Encontre todos os pedidos..."), basta executar sem a projeção:

```
db.pedidos.find({ CONTACTLASTNAME: { $regex: "Young", $options: "i" } })
```

Exercício 11

- **Status:** Correto na expressão regular.
- **Comentário:** A âncora `^P` combinada com a opção `$options: "i"` filtrou com precisão as cidades iniciadas pela letra "P" (como Paris, Porto, Pretoria e Palermo).
- **Ponto de atenção:** Mesma observação sobre o uso de projeção para omitir os demais campos do documento.

Exercício 12

- **Status:** Correto.

- **Comentário:** O intervalo delimitado por { ORDERDATE: { \$gte: "2003-07-01", \$lte: "2003-12-31" } } respeitou rigorosamente o formato de datas armazenado e os limites especificados no enunciado. (Apenas atente-se à projeção caso seja exigido o documento completo).

Pode mandar a próxima leva (do 13 em diante) quando quiser conferir!

User prompt: ex 13 a 18

Response: Todas as resoluções dos exercícios 13 a 18 estão corretas e bem estruturadas, com apenas um detalhe sobre projeção a ter em conta no Exercício 16:

Exercício 13

- **Estado:** Correto.
- **Comentário:** A agregação foi encadeada com precisão. O estágio \$lookup fez a junção com a coleção clientes, o \$unwind descompactou o array resultante e o \$project extraiu ORDERNUMBER, SALES e o campo customer_name a partir de "\$cliente.customer_name", suprimindo o _id.

Exercício 14

- **Estado:** Correto.
- **Comentário:** O comando db.pedidos.createIndex({ ORDERNUMBER: 1 }) criou com sucesso o índice no campo de pesquisa, e a execução de db.pedidos.getIndexes() confirmou a presença de ORDERNUMBER_1 ao lado do índice padrão _id_.

Exercício 15

- **Estado:** Correto.
- **Comentário:** O agrupamento composto em _id utilizando { country: "\$COUNTRY", status: "\$STATUS" } associado ao acumulador totalSales: { \$sum: "\$SALES" } cumpre rigorosamente o objetivo de totalizar as vendas por combinação de país e estado do pedido.

Exercício 16

- **Estado:** Correto na lógica de filtragem e limite.
- **Comentário:** O filtro { SALES: { \$gt: 3000 } }, a ordenação descendente .sort({ SALES: -1 }) e a limitação .limit(10) estão corretos. Como se pretendem os primeiros registros, o uso direto de .limit(10) é suficiente.
- **Ponto de atenção:** Tal como nos exercícios anteriores, incluiu uma projeção manual { _id: 0, ORDERNUMBER: 1, SALES: 1, PRODUCTLINE: 1 }. Se o critério de avaliação exigir a devolução dos pedidos inteiros conforme o enunciado ("Recupere os primeiros 10 pedidos..."), execute sem o segundo argumento do find():

```
db.pedidos.find({ SALES: { $gt: 3000 } }).sort({ SALES: -1 }).limit(10)
```

Exercício 17

- **Estado:** Correto.
- **Comentário:** O recurso ao *pipeline* de atualização no updateMany (passando o array [{ \$set: ... }]) foi aplicado de forma exemplar para permitir o uso da expressão \$multiply com os campos "\$QUANTITYORDERED" e "\$PRICEEACH". A verificação com find() demonstrou os novos valores calculados.

Exercício 18

- **Estado:** Correto.
- **Comentário:** A utilização de updateMany com o filtro { PRODUCTCODE: "S18_1342" } associada à inserção do array de objetos productDetails (com product_name, product_code e product_description) satisfaz todos os requisitos. A consulta subsequente confirmou a inclusão correta da estrutura.

Pode avançar com os exercícios 19 a 24 quando quiser validar o próximo bloco.

User prompt: ex 19 a 24

Response: As consultas dos exercícios 19 a 24 estão corretas e cumprem os requisitos com rigor:

Exercício 19

- **Estado:** Correto.
- **Comentário:** O operador \$nin foi configurado com precisão em { STATUS: { \$nin: ["Shipped", "Cancelled"] } }, excluindo com sucesso os dois estados indicados.
- **Ponto de atenção:** Caso a correção exija o documento completo conforme a instrução genérica, remova o segundo objeto de projeção do find().

Exercício 20

- **Estado:** Correto e idiomático.
- **Comentário:** A combinação do intervalo { QUANTITYORDERED: { \$gte: 20, \$lte: 40 } } com { COUNTRY: { \$ne: "USA" } } no mesmo documento de filtro aplica o operador AND implícito do MongoDB de forma limpa e eficiente.
- **Nota alternativa:** Se o docente fizer questão da utilização explícita do operador \$and (por ser o título da secção na ficha), a sintaxe equivalente seria:

```
db.pedidos.find({
  $and: [
    { QUANTITYORDERED: { $gte: 20, $lte: 40 } },
    { COUNTRY: { $ne: "USA" } }
  ]
})
```

Exercício 21

- **Estado:** Correto.
- **Comentário:** O filtro { TotalCost: { \$exists: true } } seleciona perfeitamente os documentos que possuem a propriedade calculada no Exercício 17.

Exercício 22

- **Estado:** Impecável.
- **Comentário:** A chamada db.pedidos.distinct("PRODUCTLINE") devolveu diretamente a lista com os 6 valores únicos existentes na base de dados.

Exercício 23

- **Estado:** Correto com excelente formatação.
- **Comentário:** A utilização dos acumuladores \$min: "\$PRICEEACH" e \$max: "\$PRICEEACH" agrupados por "\$PRODUCTLINE" atende à totalidade do problema. O estágio subsequente com \$project para

renomear `_id` para `PRODUCTLINE` tornou a leitura do resultado muito mais elegante.

Exercício 24

- **Estado:** Correto.
- **Comentário:** O estágio `$project` incluiu apenas `ORDERNUMBER` e `COUNTRY`, suprimiu o `_id` e calculou com exatidão os 10% através de `descontoSugerido`: `{ $multiply: ["$SALES", 0.10] }`.

Pode enviar a última secção (exercícios 25 a 30) para finalizar a verificação.

User prompt: ex 25 a 30

Response: As consultas e operações dos exercícios 25 a 30 estão corretas, com destaque para a justificação matemática do resultado final no Exercício 30:

Exercício 25

- **Estado:** Correto.
- **Comentário:** Seguiu a sugestão do enunciado ao resolver em duas etapas. Calculou a média global de vendas e utilizou o valor literal (3673.68) no filtro `{ SALES: { $gt: 3673.68 } }`, finalizando com `{ $count: "pedidosAcimaDaMedia" }`.
- **Nota contextual:** O valor de média utilizado reflete o estado real da coleção após a remoção dos registos dos EUA no Exercício 9, demonstrando coerência com a base ativa.

Exercício 26

- **Estado:** Correto.
- **Comentário:** O `$unwind`: `"$productDetails"` desagregou com precisão os itens do array adicionado no Exercício 18. O `$project` selecionou estritamente `ORDERNUMBER` e `productDetails.product_name`, suprimindo o `_id` como requerido.

Exercício 27

- **Estado:** Correto e com pipeline exemplar.
- **Comentário:** Encadeou com perfeição todas as fases necessárias:
 1. `$match` com a regex `/^2004/` para isolar o ano.
 2. `$group` por linha de produto somando `"$SALES"`.
 3. `$sort` decrescente pela receita.
 4. `$limit`: 1 para extrair apenas o líder de vendas.
 5. `$project` para renomear e formatar a saída.

Exercício 28

- **Estado:** Correto.
- **Comentário:** Utilizou `$addToSet`: `"$COUNTRY"` no agrupamento por `"$PRODUCTLINE"` para garantir elementos únicos e aplicou a expressão `$size`: `"$países"` na projeção para obter a contagem exata por linha.

Exercício 29

- **Estado:** Correto e validado tecnicamente.
- **Comentário:** A criação do índice composto `createIndex({ STATUS: 1, SALES: -1 })` seguiu a ordem e direções solicitadas. A consulta casou os campos do índice e o plano de execução (`winningPlan`) confirmou a utilização do índice através da etapa `IXSCAN` no `inputStage`.

Exercício 30

- **Estado:** Correto na sintaxe e execução (atenção à discrepância de contagem do enunciado).
- **Comentário:** A inserção com `insertMany` dos três novos documentos (10201, 10202 e 10203) foi concluída com todos os campos estruturados.
- **Ponto de atenção na contagem:** O terminal retornou **33** documentos após a inserção, enquanto o enunciado previa 38. Isto aconteceu porque:
 - A coleção iniciou com **35** documentos.
 - No Exercício 9, foram eliminados **5** documentos onde `COUNTRY: "USA"` (`deletedCount: 5`).
 - Restaram 30 documentos na coleção. Com os **3** adicionados agora, o valor real e correto é **33**.

O autor do guião calculou $35 + 3 = 38$, esquecendo-se do impacto do comando destrutivo do Exercício 9. Caso o docente ou validador automático exija obrigatoriamente o valor 38, será necessário reinserir os registos apagados ou reimportar o ficheiro `dados_pedidos.json`.

User prompt: Agora me manda os comandos que ficaram pendentes!! Para correção e tals!

Response: Os comandos revistos e prontos para correção abrangem os ajustes de nomenclatura de campos, a remoção de projeções manuais que omitiam dados solicitados nos enunciados e a sintaxe explícita para operadores lógicos.

1. Exercício 3 – Ajuste Semântico de Campo

O comando anterior utilizou `totalUnidades` para acumular valores monetários. Para refletir com exatidão o campo somado (`SALES`):

```
db.pedidos.aggregate([
  { $match: { ORDERDATE: { $gte: "2003-01-01", $lte: "2003-12-31" } } },
  { $group: { _id: null, totalVendas: { $sum: "$SALES" } } }
])
```

2. Exercícios com Remoção de Projeções Indevidas

Nos exercícios em que o enunciado solicita a recuperação dos documentos de pedidos sem especificar projeção, o segundo argumento do `find()` deve ser retirado para devolver os documentos completos:

- Exercício 6 (Recuperar todos os pedidos ordenados por `SALES` decrescente):

```
db.pedidos.find().sort({ SALES: -1 })
```

- Exercício 10 (Pedidos cujo `CONTACTLASTNAME` contenha "Young"):

```
db.pedidos.find({ CONTACTLASTNAME: { $regex: "Young", $options: "i" } })
```

- Exercício 11 (Pedidos em que `CITY` comece pela letra "P"):

```
db.pedidos.find({ CITY: { $regex: "^P", $options: "i" } })
```

- Exercício 12 (Pedidos realizados entre 01/07/2003 e 31/12/2003):

```
db.pedidos.find({ ORDERDATE: { $gte: "2003-07-01", $lte: "2003-12-31" } })
```

- Exercício 16 (Primeiros 10 pedidos com SALES > 3000 ordenados decrescente):

```
db.pedidos.find({ SALES: { $gt: 3000 } }).sort({ SALES: -1 }).limit(10)
```

- Exercício 19 (Pedidos com STATUS que não seja "Shipped" nem "Cancelled"):

```
db.pedidos.find({ STATUS: { $nin: ["Shipped", "Cancelled"] } })
```

3. Exercício 20 – Versão com \$and Explícito

A solução com AND implícito é plenamente válida, mas caso o avaliador exija a aplicação explícita do operador demonstrado na secção da ficha:

```
db.pedidos.find({
  $and: [
    { QUANTITYORDERED: { $gte: 20, $lte: 40 } },
    { COUNTRY: { $ne: "USA" } }
  ]
})
```

4. Exercício 30 – Reposição dos Dados para Obter os 38 Documentos

A contagem final resultou em 33 documentos porque 5 registos dos Estados Unidos foram eliminados no Exercício 9. Se a correção for automatizada e validar rigidamente o número 38, importe novamente a base no terminal do sistema operativo antes de rodar o insertMany:

```
mongoimport --db loja --collection pedidos --file dados_pedidos.json --jsonArray --drop
```

De seguida, execute a inserção dos 3 pedidos no mongosh:

```
db.pedidos.insertMany([
  {
    ORDERNUMBER: 10201,
    QUANTITYORDERED: 25,
    PRICEEACH: 80,
    SALES: 2000,
    STATUS: "In Process",
    ORDERDATE: "2005-11-02",
    PRODUCTLINE: "Trucks",
    PRODUCTCODE: "S18_1342",
    CUSTOMER_ID: 2,
    CONTACTLASTNAME: "Silva",
    CITY: "São Paulo",
    COUNTRY: "Brazil"
  },
  {
    ORDERNUMBER: 10202,
    QUANTITYORDERED: 15,
    PRICEEACH: 120,
    SALES: 1800,
    STATUS: "Shipped",
    ORDERDATE: "2005-11-03",
    PRODUCTLINE: "Motorcycles",
    PRODUCTCODE: "S10_1678",
    CUSTOMER_ID: 3,
    CONTACTLASTNAME: "Martins",
    CITY: "Lisbon",
    COUNTRY: "Portugal"
  },
  {
    ORDERNUMBER: 10203,
    QUANTITYORDERED: 50,
```

```
PRICEEACH: 60,  
SALES: 3000,  
STATUS: "Cancelled",  
ORDERDATE: "2005-11-04",  
PRODUCTLINE: "Classic Cars",  
PRODUCTCODE: "S24_2000",  
CUSTOMER_ID: 4,  
CONTACTLASTNAME: "Fernandez",  
CITY: "Madrid",  
COUNTRY: "Spain"  
}  
])  
  
db.pedidos.countDocuments() // Retornará 38
```

User prompt: CONSTRUA UM DOC sendo uma revisão de todos os comandos que vimos e utilizamos nesse DOC!!! Quero fazer algo completo bem explicado! Pode fazer um doc google mesmo

Response: GUIA DE REVISÃO E REFERÊNCIA: MONGODB & AGGREGATION FRAMEWORK

Este documento reúne a arquitetura, sintaxe, boas práticas e casos práticos de todos os comandos abordados na série de exercícios sobre a base de dados loja (coleções pedidos e clientes).

1. Configuração de Ambiente e Utilitários de Shell

Importação de Coleções (CLI)

A carga de coleções estruturadas em arquivos JSON é executada fora do mongosh, diretamente no terminal do sistema operacional:

```
# Importação da coleção de pedidos  
mongoimport -db loja -collection pedidos -file dados_pedidos.json -jsonArray  
  
# Importação da coleção de clientes  
mongoimport -db loja -collection clientes -file clientes.json -jsonArray
```

- `--db / -db`: Define o banco de dados de destino.
- `-collection`: Nomeia a coleção que receberá a carga.
- `-jsonArray`: Informa ao utilitário que o arquivo de origem contém um array de objetos JSON.

Gestão Básica no mongosh

```
// Selecionar o banco de trabalho  
use loja  
  
// Validação volumétrica  
db.pedidos.countDocuments() // Retorno esperado inicial: 35  
db.clientes.countDocuments() // Retorno esperado inicial: 30
```

2. Operações de Consulta Básica e Filtros (find)

O método `db.colecao.find(filtro, projecao)` recebe dois parâmetros: o critério de busca e os campos a serem devolvidos.

Operadores de Comparação Numérica e Relacional

O MongoDB disponibiliza operadores relacionais com o prefixo \$: \$gt (maior), \$gte (maior ou igual), \$lt (menor), \$lte (menor ou igual), \$eq (igual) e \$ne (diferente).

- Exercício 1 – Filtro relacional simples (\$gt):

```
// Recupera todos os pedidos com quantidade pedida superior a 50
db.pedidos.find( { QUANTITYORDERED: { $gt: 50 } } )
```

Projeção de Campos

A projeção controla o tráfego de dados entre o servidor e o cliente. O valor 1 inclui o atributo, 0 o exclui explicitamente. Por padrão, o atributo primário _id sempre é incluído, a menos que seja forçado para 0.

- Exercício 2 – Projeção de atributos específicos:

```
// Retorna apenas número do pedido, vendas e status, suprimindo o _id
db.pedidos.find( {}, { ORDERNUMBER: 1, SALES: 1, STATUS: 1, _id: 0 } )
```

Expressões Regulares (\$regex)

O operador \$regex permite varreduras textuais baseadas em expressões regulares padrão PCRE. O modificador \$options: "i" aplica a busca indiferente a maiúsculas e minúsculas (*case-insensitive*).

- Exercício 10 – Pesquisa por substring (*contains*):

```
// Localiza pedidos cujo sobrenome do contato contenha o fragmento "Young"
db.pedidos.find( { CONTACTLASTNAME: { $regex: "Young", $options: "i" } } )
```

- Exercício 11 – Expressão regular com âncora de início (^):

```
// Filtra pedidos em cidades cujo nome inicie rigorosamente pela letra "P"
db.pedidos.find( { CITY: { $regex: "^P", $options: "i" } } )
```

Intervalos de Datas e Dados Temporais

A combinação de limites inferiores (\$gte) e superiores (\$lte) sobre o mesmo campo permite filtrar períodos no padrão ISO 8601 (YYYY-MM-DD).

- Exercício 12 – Filtragem por intervalo de datas:

```
// Retorna os pedidos feitos no segundo semestre de 2003
db.pedidos.find( {
  ORDERDATE: { $gte: "2003-07-01", $lte: "2003-12-31" }
} )
```

Operadores de Lista e Lógica Booleana

- \$in / \$nin: Avaliam se um valor está ou não contido em um array de parâmetros.
- \$and / \$or: Aplicam lógica booleana explícita sobre um conjunto de condições.
- Exercício 19 – Operador de não-inclusão (\$nin):

```
// Pedidos cujo STATUS não seja "Shipped" e nem "Cancelled"
db.pedidos.find( { STATUS: { $nin: ["Shipped", "Cancelled"] } } )
```

- Exercício 20 – Conjunção lógica (\$and / Condições simultâneas):

```
// Pedidos com quantidade entre 20 e 40 e país de envio fora dos EUA
db.pedidos.find( {
  $and: [
```

```

    { QUANTITYORDERED: { $gte: 20, $lte: 40 } },
    { COUNTRY: { $ne: "USA" } }
  ]
})

```

Checação de Estrutura e Distinção

- Exercício 21 – Verificação de existência de campos (\$exists):

```

// Localiza documentos que possuam o campo TotalCost persistido
db.pedidos.find({ TotalCost: { $exists: true } })

```

- Exercício 22 – Listagem de valores distintos (distinct):

```

// Lista todos os nomes únicos das linhas de produto da coleção
db.pedidos.distinct("PRODUCTLINE")

```

3. Manipulação e Modificação de Documentos (CRUD)

Atualizações Pontuais e em Massa

A modificação atômica de propriedades requer o operador \$set para evitar a substituição integral do documento.

- Exercício 7 – Atualização de registro único (updateOne):

```

// Modifica o status de um pedido pontual
db.pedidos.updateOne(
  { ORDERNUMBER: 10107 },
  { $set: { STATUS: "Delivered" } }
)

```

- Exercício 8 – Atualização condicional em lote (updateMany):

```

// Altera o status para "Processing" em pedidos de baixo volume (< 30)
db.pedidos.updateMany(
  { QUANTITYORDERED: { $lt: 30 } },
  { $set: { STATUS: "Processing" } }
)

```

Atualizações Avançadas com Pipeline de Agregação

Ao envolver o segundo parâmetro do updateMany em colchetes [...], ativa-se o pipeline de atualização, permitindo que campos existentes sirvam de entrada para operações aritméticas em novos atributos.

- Exercício 17 – Campo calculado persistido (\$multiply):

```

// Calcula e persiste o campo TotalCost como (QUANTITYORDERED * PRICEEACH)
db.pedidos.updateMany(
  {},
  [
    {
      $set: {
        TotalCost: { $multiply: ["$QUANTITYORDERED", "$PRICEEACH"] }
      }
    }
  ]
)

```

Persistência de Subdocumentos e Estruturas Aninhadas

- Exercício 18 – Inclusão de arrays de objetos:

```
// Insere um array de detalhes de produto em todos os pedidos de um código específico
db.pedidos.updateMany(
  { PRODUCTCODE: "S18_1342" },
  {
    $set: {
      productDetails: [
        {
          product_name: "Truck Model",
          product_code: "S18_1342",
          product_description: "Linha de produtos Trucks"
        }
      ]
    }
  }
)
```

Exclusão Controlada de Registros

A exclusão irreversível exige validação prévia para auditoria do total de documentos atingidos.

- Exercício 9 – Exclusão em lote (deleteMany):

```
// 1. Auditoria prévia
db.pedidos.countDocuments({ COUNTRY: "USA" })

// 2. Execução da remoção
db.pedidos.deleteMany({ COUNTRY: "USA" })

// 3. Validação de segurança
db.pedidos.countDocuments({ COUNTRY: "USA" }) // Deve retornar 0
```

Inserção de Registros

- Exercício 30 – Inserção múltipla (insertMany):

```
// Insere múltiplos documentos de pedidos gerando IDs automáticos
db.pedidos.insertMany([
  {
    ORDERNUMBER: 10201,
    QUANTITYORDERED: 25,
    PRICEEACH: 80,
    SALES: 2000,
    STATUS: "In Process",
    ORDERDATE: "2005-11-02",
    PRODUCTLINE: "Trucks",
    PRODUCTCODE: "S18_1342",
    CUSTOMER_ID: 2,
    CONTACTLASTNAME: "Silva",
    CITY: "São Paulo",
    COUNTRY: "Brazil"
  },
  {
    ORDERNUMBER: 10202,
    QUANTITYORDERED: 15,
    PRICEEACH: 120,
    SALES: 1800,
    STATUS: "Shipped",
    ORDERDATE: "2005-11-03",
    PRODUCTLINE: "Motorcycles",
  }
])
```

```

    PRODUCTCODE: "S10_1678",
    CUSTOMER_ID: 3,
    CONTACTLASTNAME: "Martins",
    CITY: "Lisbon",
    COUNTRY: "Portugal"
  },
  {
    ORDERNUMBER: 10203,
    QUANTITYORDERED: 50,
    PRICEEACH: 60,
    SALES: 3000,
    STATUS: "Cancelled",
    ORDERDATE: "2005-11-04",
    PRODUCTLINE: "Classic Cars",
    PRODUCTCODE: "S24_2000",
    CUSTOMER_ID: 4,
    CONTACTLASTNAME: "Fernandez",
    CITY: "Madrid",
    COUNTRY: "Spain"
  }
]

```

4. O Framework de Agregação (aggregate)

O método `aggregate([ estágio1, estágio2, ... ])` processa documentos sequencialmente em estágios encadeados (*pipeline*).

Estágios Estruturais

Estágio	Finalidade Técnica
<code>\$match</code>	Filtra os documentos de entrada (análogo ao <code>WHERE</code> relacional).
<code>\$group</code>	Agrupar documentos por uma chave <code>_id</code> e calcula métricas agregadas.
<code>\$project</code>	Modifica formatos, renomeia, inclui, suprime ou calcula campos.
<code>\$sort</code>	Ordena o fluxo de documentos (1: crescente, -1: decrescente).
<code>\$limit</code>	Restringe o volume máximo de documentos passados adiante.
<code>\$skip</code>	Descarta os primeiros N documentos do fluxo de saída.
<code>\$unwind</code>	Descompacta arrays, criando um documento por elemento.
<code>\$lookup</code>	Realiza junção lateral (<i>Left Outer Join</i>) com outra coleção.
<code>\$count</code>	Devolve a contagem inteira de documentos sob um novo atributo.

Agrupamento Global e Acumuladores

Quando a chave `_id` de um agrupamento é definida como `null`, o MongoDB consolida todos os documentos recebidos em uma única partição.

- Exercício 3 – Agrupamento global com filtro prévio (`$sum`):

```

db.pedidos.aggregate([
  { $match: { ORDERDATE: { $gte: "2003-01-01", $lte: "2003-12-31" } } },
  { $group: { _id: null, totalVendas: { $sum: "$SALES" } } }
])

```

- Exercício 5 – Média aritmética (\$avg) com arredondamento (\$round):

```
db.pedidos.aggregate([
  { $group: { _id: null, precoMedio: { $avg: "$SALES" } } },
  { $project: { _id: 0, precoMedioFormatado: { $round: ["$precoMedio", 2] } } }
])
```

Agrupamento por Chaves Simples e Compostas

Para criar subgrupos, referencia-se o caminho do atributo precedido por \$. Para agrupar por múltiplos eixos, atribui-se um subdocumento ao _id.

- Exercício 4 – Agrupamento por chave simples:

```
db.pedidos.aggregate([
  { $group: { _id: "$COUNTRY", totalCountry: { $sum: "$SALES" } } }
])
```

- Exercício 15 – Agrupamento por chave composta:

```
db.pedidos.aggregate([
  {
    $group: {
      _id: { country: "$COUNTRY", status: "$STATUS" },
      totalSales: { $sum: "$SALES" }
    }
  }
])
```

- Exercício 23 – Acumuladores extremos (\$min e \$max):

```
db.pedidos.aggregate([
  {
    $group: {
      _id: "$PRODUCTLINE",
      minPrice: { $min: "$PRICEEACH" },
      maxPrice: { $max: "$PRICEEACH" }
    }
  },
  {
    $project: {
      _id: 0,
      PRODUCTLINE: "$_id",
      minPrice: 1,
      maxPrice: 1
    }
  }
])
```

Junção de Coleções e Achatamento de Dados (\$lookup e \$unwind)

- Exercício 13 – Junção Relacional (*LEFT JOIN*):

```
db.pedidos.aggregate([
  {
    $lookup: {
      from: "clientes",
      localField: "CUSTOMER_ID",
      foreignField: "customer_id",
      as: "cliente"
    }
  },
  {
    $unwind: "$cliente"
  }
])
```

```

    { $unwind: "$cliente" },
    {
      $project: {
        _id: 0,
        ORDERNUMBER: 1,
        SALES: 1,
        customer_name: "$cliente.customer_name"
      }
    }
  ]
})

```

- Exercício 26 – Achatamento de arrays (*Unwind*):

```

db.pedidos.aggregate([
  { $unwind: "$productDetails" },
  {
    $project: {
      _id: 0,
      ORDERNUMBER: 1,
      "productDetails.product_name": 1
    }
  }
])

```

Campos Calculados em Projeção

- Exercício 24 – Projeção aritmética com expressões:

```

db.pedidos.aggregate([
  {
    $project: {
      _id: 0,
      ORDERNUMBER: 1,
      COUNTRY: 1,
      descontoSugerido: { $multiply: ["$SALES", 0.10] }
    }
  }
])

```

Agrupamento com Conjuntos e Contagem Única

- Exercício 28 – Operadores \$addToSet e \$size:

```

db.pedidos.aggregate([
  {
    $group: {
      _id: "$PRODUCTLINE",
      paises: { $addToSet: "$COUNTRY" }
    }
  },
  {
    $project: {
      _id: 0,
      PRODUCTLINE: "$_id",
      nPaises: { $size: "$paises" }
    }
  }
])

```

Pipelines Multi-estágio Complexos

- Exercício 27 – Encadeamento analítico completo:

```
db.pedidos.aggregate([
  // 1. Filtrar vendas ocorridas em 2004
  { $match: { ORDERDATE: { $regex: "^2004" } } },

  // 2. Agrupar receita por linha de produtos
  {
    $group: {
      _id: "$PRODUCTLINE",
      receitaTotal: { $sum: "$SALES" }
    }
  },

  // 3. Ordenar do maior para o menor faturamento
  { $sort: { receitaTotal: -1 } },

  // 4. Extrair o líder absoluto
  { $limit: 1 },

  // 5. Ajustar a formatação final
  {
    $project: {
      _id: 0,
      PRODUCTLINE: "$_id",
      receitaTotal: 1
    }
  }
])
```

- Exercício 25 – Contagem de documentos agregados (\$count):

```
// Localiza pedidos com vendas acima da média da base (ex: 3673.68) e conta o total
db.pedidos.aggregate([
  { $match: { SALES: { $gt: 3673.68 } } },
  { $count: "pedidosAcimaDaMedia" }
])
```

5. Paginação, Ordenação e Performance de Índices

Métodos de Cursor: .sort(), .skip() e .limit()

Para paginação eficiente, métodos de cursor encadeiam-se diretamente sobre o find().

- Exercício 6 – Ordenação pura:

```
db.pedidos.find().sort({ SALES: -1 })
```

- Exercício 16 – Paginação com filtro e limite:

```
// Primeiro lote de 10 pedidos de alto valor
db.pedidos.find({ SALES: { $gt: 3000 } })
  .sort({ SALES: -1 })
  .limit(10)

// Segundo lote (página 2, ignorando os 10 primeiros):
db.pedidos.find({ SALES: { $gt: 3000 } })
  .sort({ SALES: -1 })
  .skip(10)
  .limit(10)
```

Criação e Inspeção de Índices

Índices evitam a varredura completa da coleção (*COLLSCAN*), reduzindo o custo de I/O em leitura.

- Exercício 14 – Índice simples:

```
// Cria índice crescente no número do pedido
db.pedidos.createIndex({ ORDERNUMBER: 1 })
```

```
// Consulta os índices presentes na coleção
db.pedidos.getIndexes()
```

- Exercício 29 – Índice composto e plano de execução:

```
// 1. Criação do índice com ordenações específicas por atributo
db.pedidos.createIndex({ STATUS: 1, SALES: -1 })
```

```
// 2. Consulta com auditoria de plano de execução
db.pedidos.find({ STATUS: "Shipped" })
  .sort({ SALES: -1 })
  .explain("executionStats")
```

Verificação no Plano de Execução (explain): O campo `winningPlan.inputStage.stage` deve apresentar *IXSCAN* (varredura pelo índice). Se apresentar *COLLSCAN*, o MongoDB precisou ler todos os documentos em disco porque o índice não cobriu a consulta.

6. Boas Práticas de Engenharia em MongoDB

- **Validação Prévia de Operações Destrutivas:** Antes de rodar `deleteMany` ou `updateMany`, utilize o mesmo objeto de filtro dentro de `countDocuments(filtro)` ou `find(filtro)` para auditar quais documentos serão atingidos.
- **Modularização de Pipelines de Agregação:** Para pipelines extensos, declare a sequência de estágios em uma constante no terminal para facilitar testes e reutilização:

```
const pipeline = [
  { $match: { STATUS: "Shipped" } },
  { $group: { _id: "$COUNTRY", total: { $sum: "$SALES" } } },
  { $sort: { total: -1 } }
];
db.pedidos.aggregate(pipeline);
```

- **Ordenação de Campos em Índices Compostos:** Em índices com múltiplos campos, ordene os atributos seguindo a regra ESR (*Equality, Sort, Range*): igualdade exata primeiro, ordenação depois, e intervalos por último.

User prompt: Pode mandar também uma revisão geral do que vamos precisar usar no MongoDB?? Tipo, aprofundar alguns diversos conceitos básicos de comandos no MongoDB

Response: O domínio do MongoDB exige a compreensão de como o motor BSON processa estruturas aninhadas, avalia operadores atômicos e percorre índices em memória. Abaixo estão os pilares conceituais essenciais que complementam as consultas do dia a dia.

1. Modelo de Dados BSON e Notação de Ponto (*Dot Notation*)

Diferente de bancos relacionais que normalizam dados em tabelas rígidas, o MongoDB armazena documentos em **BSON** (*Binary JSON*). O BSON suporta tipos de dados adicionais como `ObjectId`, tipos de data (`ISODate`), inteiros de 64 bits e números decimais de alta precisão.

Relacionamento: Embutir (*Embed*) vs. Referenciar (*Reference*)

- **Embutir (*Embedded Documents*):** Armazena subdocumentos ou arrays diretamente dentro do documento pai (relação 1 : 1 ou 1 : N com poucos filhos, como o array `productDetails`). Proporciona leituras atômicas e alta performance de I/O em uma única operação.
- **Referenciar (*Reference*):** Guarda apenas o identificador (`CUSTOMER_ID`) apontando para outra coleção (`clientes`), exigindo junções via `$lookup` ou múltiplas consultas pela aplicação (relação 1 : N grande ou $N : N$).

Acesso a Estruturas Profundas com *Dot Notation*

Para consultar ou atualizar campos aninhados ou elementos de arrays, utiliza-se a notação de ponto delimitada por aspas:

- **Em Subdocumentos:** `"endereco.cidade"` acessa o campo `cidade` dentro do subdocumento `endereco`.
- **Em Arrays de Objetos:** `"productDetails.product_name"` avalia se *pelo menos um* objeto do array contém a propriedade especificada.
- **Por Índice de Array:** `"itens.0.preco"` acessa diretamente o preço do primeiro item do array.

2. Operadores de Consulta e Filtragem em Arrays

Além dos operadores relacionais comuns (`$gt`, `$lte`, `$in`), cenários reais exigem consultas precisas sobre listas e tipos:

Operadores Específicos para Arrays

- **`$all`:** O array no documento precisa conter *todos* os elementos listados, independentemente da ordem.

```
// Retorna pedidos que contenham ambas as tags simultaneamente
db.pedidos.find({ tags: { $all: ["importado", "urgente"] } })
```

- **`$elemMatch`:** Exige que um **único elemento** do array satisfaça múltiplos critérios simultaneamente. É crucial para evitar falsos positivos ao consultar arrays de objetos:

```
// Garante que o MESMO produto no array tenha código X e preço > 100
db.pedidos.find({
  productDetails: {
    $elemMatch: { product_code: "S18_1342", price: { $gt: 100 } }
  }
})
```

- **`$size`:** Realiza correspondência baseada no comprimento exato de um array (não aceita operadores relacionais como `$gt` diretamente no `find()`).

Checagem de Tipos e Existência

- **`$type`:** Filtra por tipo de dado BSON (útil para detectar campos com inconsistências de casting entre string e numérico):

```
db.pedidos.find({ SALES: { $type: "double" } })
```

- **`$exists`:** Avalia a presença da chave no documento (independente do valor ser `null`).

3. Mecanismos de Modificação Atômica (Atualizações)

As mutações no MongoDB devem ser idempotentes e atômicas a nível de documento. A ausência de operadores atômicos em comandos de atualização pode sobrescrever acidentalmente o documento inteiro.

Operador	Ação Técnica	Exemplo
\$set	Cria ou substitui o valor de uma chave específica sem afetar as demais.	{ \$set: { STATUS: "Shipped" } }
\$unset	Remove completamente o campo do documento.	{ \$unset: { TotalCost: "" } }
\$inc	Incrementa ou decrementa valores numéricos de forma atômica.	{ \$inc: { QUANTITYORDERED: 5, estoque: -5 } }
\$push	Adiciona um novo elemento ao final de um array.	{ \$push: { logs: "Pedido despachado" } }
\$addToSet	Insere um item no array apenas se ele ainda não existir lá dentro (garante unicidade).	{ \$addToSet: { tags: "promocional" } }
\$pull	Remove todos os elementos de um array que atendam a uma condição.	{ \$pull: { tags: "obsoleto" } }
\$currentDate	Atualiza o campo com a data/hora atual do servidor.	{ \$currentDate: { updatedAt: true } }

O Modificador upsert

Quando ativado no `updateOne` ou `updateMany`, o parâmetro `{ upsert: true }` instrui o banco a criar um novo documento caso o filtro não encontre nenhuma correspondência:

```
db.pedidos.updateOne(
  { ORDERNUMBER: 10500 },
  { $set: { STATUS: "New", SALES: 450 } },
  { upsert: true }
)
```

4. Otimização, Índices e a Regra ESR

Sem índices adequados, qualquer operação de busca exige a leitura de cada documento do disco para a memória (*Collection Scan* ou *COLLSCAN*). Os índices B-Tree ordenam ponteiros para os documentos, permitindo buscas logarítmicas (*Index Scan* ou *IXSCAN*).

A Regra ESR para Índices Compostos

Ao criar um índice com múltiplos campos (como no Exercício 29), a ordem dos atributos deve seguir a convenção **ESR**:

1. **E (Equality)**: Campos filtrados por valores exatos vêm no início do índice (`STATUS: 1`).
2. **S (Sort)**: Campos utilizados para ordenação vêm no meio (`SALES: -1`). Isso permite que o motor aproveite a árvore já ordenada e evite ordenar dados em memória (*blocking sort*).
3. **R (Range)**: Campos consultados por intervalos (`$gt`, `$lt`, `$gte`) devem vir por último (`ORDERDATE: 1`).

```
// Exemplo aplicando ESR: Equality (STATUS), Sort (SALES), Range (ORDERDATE)
db.pedidos.createIndex({ STATUS: 1, SALES: -1, ORDERDATE: 1 })
```

Leitura do Plano de Execução (`explain("executionStats")`)

A inspeção via `.explain("executionStats")` fornece as métricas de saúde da consulta:

- **totalDocsExamined vs. nReturned**: Em uma consulta bem otimizada, a razão entre documentos examinados e retornados deve tender a 1 : 1. Se o banco precisou examinar centenas de documentos

para retornar apenas 2, o índice está ineficiente ou ausente.

- **totalKeysExamined:** Quantidade de entradas no índice avaliadas durante o percurso.
 - **stage: "FETCH":** Fase que busca o documento completo no disco após localizar sua chave no índice. Se uma consulta utiliza apenas campos que já existem no próprio índice (consulta coberta ou *Covered Query*), a fase FETCH é eliminada e o MongoDB responde direto da memória cache.
-